

Gremlin Stealer's Evolved Tactics: Hiding in Plain Sight With Resource Files

 unit42.paloaltonetworks.com/gremlin-stealer-evolution

Pranay Kumar Chhapparwal, Mark Lim

May 15, 2026

Executive Summary

This article examines new obfuscation techniques the Gremlin stealer malware uses to conceal malicious payloads within embedded resources. We analyze a variant protected by a sophisticated commercial packing utility that employs instruction virtualization, transforming the original code into a custom, non-standard bytecode executed by a private virtual machine.

Gremlin stealer siphons sensitive information from compromised systems and exfiltrates it to attacker-controlled servers for potential publication or sale. It targets web browsers, system clipboard and local storage to exfiltrate sensitive information like:

- Payment card details
- Browser cookies
- Session tokens
- Cryptocurrency wallet data
- FTP and VPN credentials

This threat has rapidly evolved, incorporating new anti-analysis safeguards into recent builds.

Palo Alto Networks customers are better protected from Gremlin Stealer through our Network Security solutions and Cortex line of products, including:

- Cortex XDR and XSIAM
- Advanced WildFire
- Advanced Threat Prevention
- Advanced URL Filtering and Advanced DNS Security

If you think you might have been compromised or have an urgent matter, contact the Unit 42 Incident Response team.

Related Unit 42 Topics **Infostealer, Telegram, Cryptocurrency**

New Gremlin Site for Publishing Data

Using data from our internal threat intelligence, we identified a new Gremlin stealer variant. This variant exfiltrates stolen data to a newly deployed site at hxxp[:]194.87.92[.]109 as shown in Figure 1.

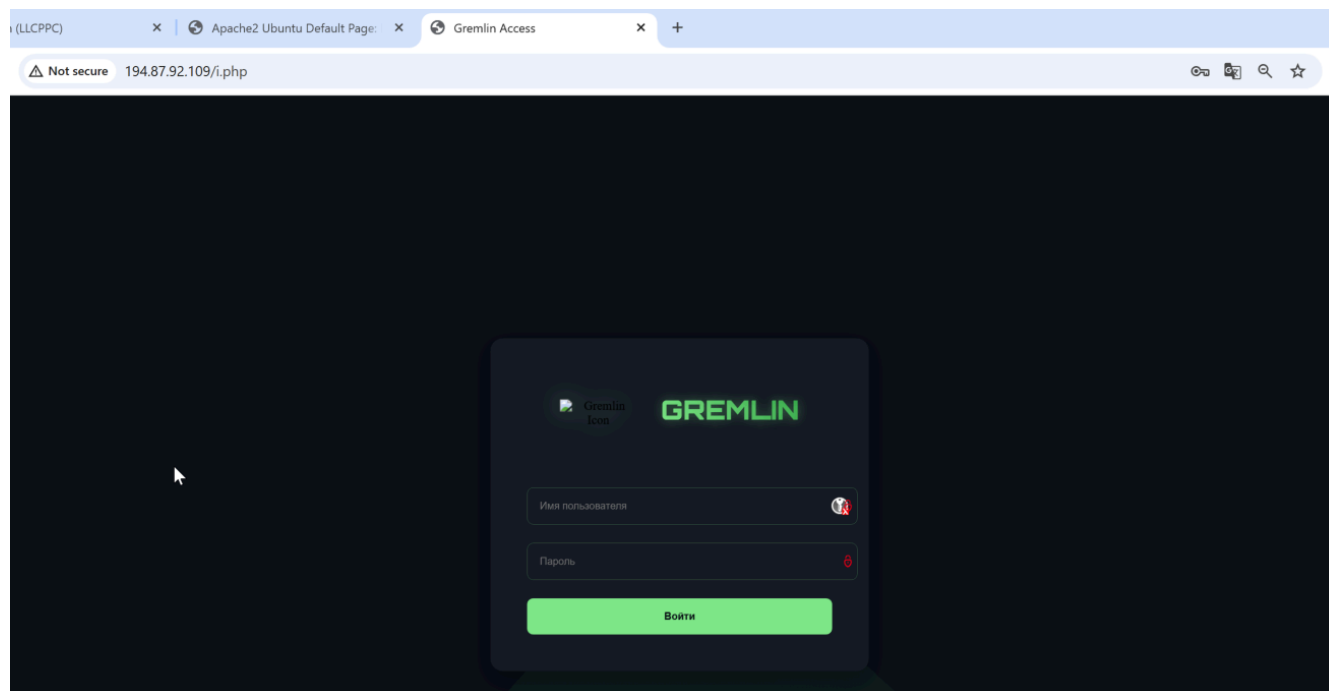


Figure 1. New Gremlin site.

At the time of discovery, VirusTotal showed zero detection for this new Gremlin site hxxp[:]194.87.92[.]109, its associated URLs or any retrieved artifacts. There were no block list entries, community reports or malicious categorizations as shown in Figure 2.

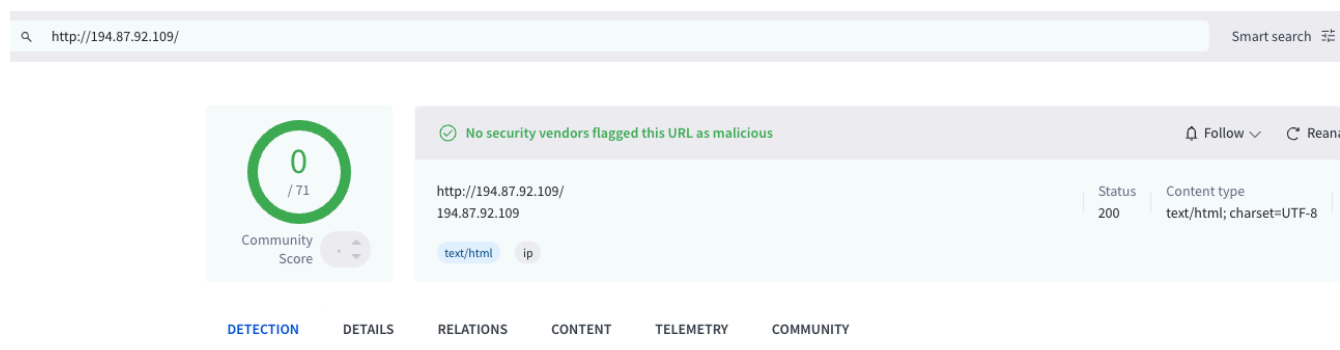


Figure 2. Gremlin Stealer's new site detection on VirusTotal.

After data theft, the malware bundles harvested artifacts into a ZIP archive, including:

- Browser cookies
- Session tokens
- Clipboard contents
- Cryptocurrency wallet data

- FTP and VPN credentials

The malware names the file using the victim's public IP address to identify the source, and then uploads it to the attacker-controlled site, as shown in Figure 3.

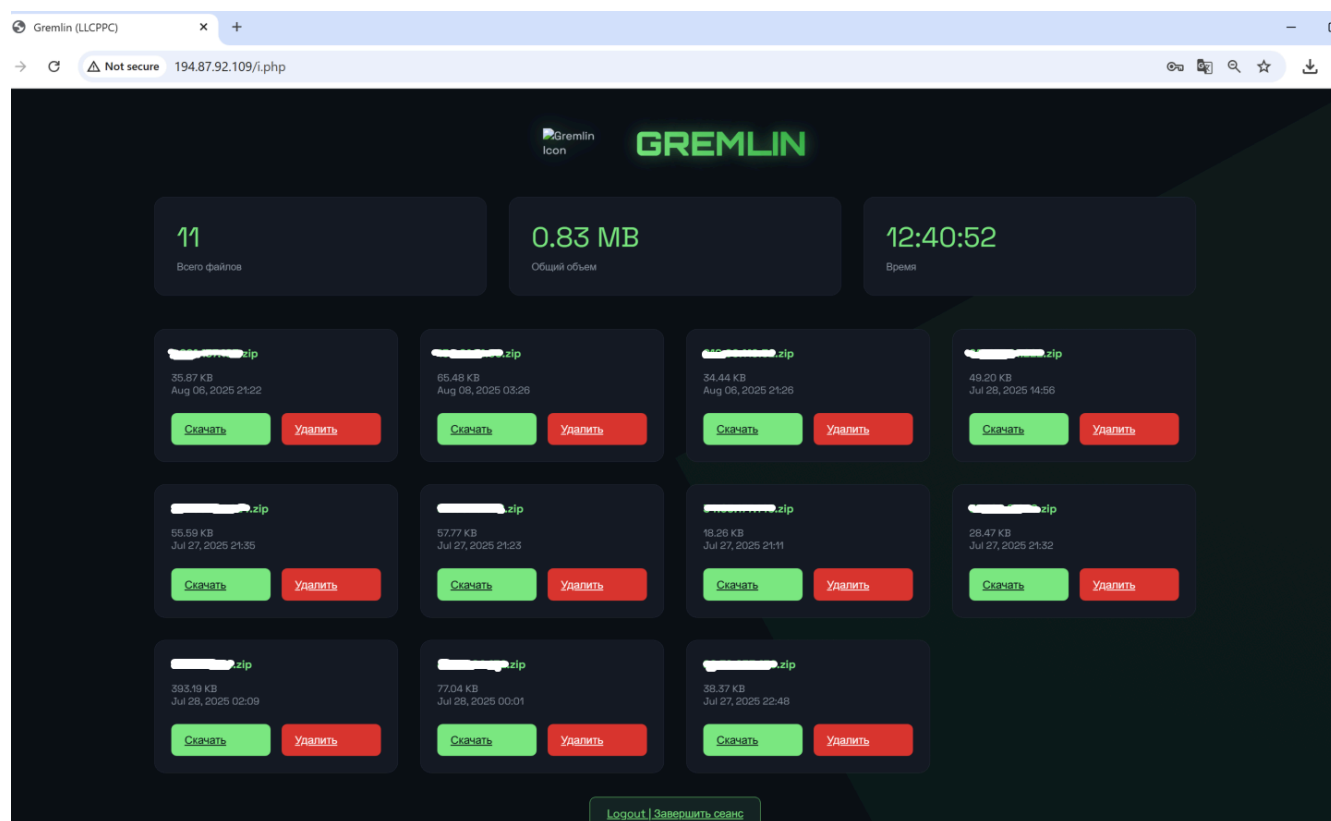


Figure 3. Gremlin site published data.

Technical Analysis

In this section, we present a comparative analysis of older and newer Gremlin stealer variants, highlighting the key changes and describing our process for extracting the final-stage payloads.

Hiding Payload in Resource

The latest iteration of the Gremlin stealer has an increased focus on stealth, specifically designed to evade static analysis tools. In this version, the malware authors have shifted the malicious payload into the .NET Resource section, masking it with XOR encoding to bypass signature-based detection and heuristic scanning.

Figure 4 shows how the resource section appears as an opaque block of data, hiding strings and API calls that would otherwise trigger alerts.

ab0fa760bd037a95c4dee431e649e

Version Info

Configuration Files

.NET Resources

wg.resources

resource

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
00000000	C5	D6	C1	C0	42	86	61	1B	5F	2D	7D	16	6A	F2	1C	3E	ÃÖÅAB æ0_-}0 jò >
00000010	8A	59	B8	57	4C	6E	D8	34	9A	5F	2D	7D	21	6A	F2	1C	Y,WLn04 _-}1jò
00000020	AE	EE	B4	FB	A0	A3	8D	51	34	47	58	2D	7D	22	6A	F2	0i'û £ Q4GX-}"jò
00000030	1C	1A	36	4B	7E	08	6D	56	FD	8D	DE	58	2D	7D	74	6A	0 6K'0 mVý bX-}tj
00000040	F2	1C	D0	65	09	1B	BD	E6	CC	67	22	53	59	2D	7D	5C	ò De.0 ælg"SY-}\
00000050	6A	F2	1C	9F	F3	F4	B9	AE	81	5D	FD	13	B2	59	2D	7D	jò óó> jý0 ^Y-
00000060	20	6A	F2	1C	A5	BC	58	F1	7D	4F	7F	17	E6	0B	5A	2D	.jò ¼4Xñ}0 0 æ0 Z-
00000070	7D	73	6A	F2	1C	48	E3	A0	93	2A	E4	AB	1D	8A	D9	5A	-}sjò HÅ *a« ÜZ
00000080	2D	7D	1E	6A	F2	1C	C6	2A	DC	35	21	A3	74	A4	BF	61	-}jò Å*U5 æt0za
00000090	5B	2D	7D	28	6A	F2	1C	A5	C0	C1	B8	7A	D8	AD	D3	F2	[-}(jò ¼AA,z0-0ò
000000A0	4D	5B	2D	7D	25	6A	F2	1C	AC	28	DF	D2	26	3F	2B	EF	M[-}%jò -(B0&?+i
000000B0	6B	25	5B	2D	7D	27	6A	F2	1C	E1	8E	BE	92	4C	52	2F	k%[-}'jò æ ¼'LR/
000000C0	61	2F	C9	5B	2D	7D	28	6A	F2	1C	A4	D8	C6	2B	7E	80	a/E[-}(jò ¼0Æ+^
000000D0	95	C9	0E	B7	5B	2D	7D	29	6A	F2	1C	8F	87	DE	16	60	È0 [-}jò F0`
000000E0	D6	B0	70	F7	4F	54	2D	7D	31	6A	F2	1C	2B	9F	9A	F2	0'p=0T-}1jò + 0
000000F0	62	0D	28	C9	3D	58	54	2D	7D	38	6A	F2	1C	8A	5D	67	b.(E=XT-}8jò lg
00000100	5E	43	02	15	BA	87	37	54	2D	7D	44	6A	F2	1C	31	F0	^C 0 0'7T-}Djò 18
00000110	16	B9	2C	79	B5	4C	23	C9	54	2D	7D	3E	6A	F2	1C	B7	0'1,yuL#ET-}>jò .
00000120	70	8F	98	E7	9C	84	37	DB	AA	54	2D	7D	29	6A	F2	1C	p ç 70^T-}jò
00000130	E6	ED	CB	8F	5D	E3	B1	12	20	BB	54	2D	7D	2C	6A	F2	æiE ã0 .>T-}jò
00000140	1C	85	BD	10	F5	4A	53	91	81	51	8D	54	2D	7D	2C	6A	¼0 ðJS' Q T-}.j
00000150	F2	1C	CA	F3	AE	9D	59	8C	DE	86	C7	68	55	2D	7D	25	ò È00 Y B çHu-}%
00000160	6A	F2	1C	13	48	1A	25	9C	99	3A	0D	0C	43	55	2D	7D	jò 0 H0 % : CU-
00000170	43	6A	F2	1C	3A	02	42	CE	2B	31	31	6E	83	21	55	2D	Cjò : B +11n U-
00000180	7D	0E	6A	F2	1C	55	2E	60	0C	2E	5A	74	8F	4D	D2	55	-}0jò U. . Zt M0U
00000190	2D	7D	2E	6A	F2	1C	17	59	29	69	E7	17	E8	B7	D9	99	-}jò 0 Y ic0è.0
000001A0	55	2D	7D	2E	6A	F2	1C	41	F4	39	FA	0C	A7	CE	3F	DE	U-}.jò Å09ú SÍ?b
000001B0	62	56	2D	7D	2E	6A	F2	1C	B8	AE	59	DD	6D	16	77	55	bV-}.jò 0YÝm0wU
000001C0	15	26	56	2D	7D	2B	6A	F2	1C	C2	D3	50	A0	0B	B1	5B	0&V-}+jò ÁÓP 0±
000001D0	2A	09	CA	56	2D	7D	2E	6A	F2	1C	39	97	45	7C	D4	DE	*.EV-}.jò 9 E ÓP
000001E0	C1	A4	F7	91	56	2D	7D	2E	6A	F2	1C	F1	0B	84	73	AD	Á0- V-}.jò ñ0 s-
000001F0	FD	FE	B3	64	26	57	2D	7D	27	6A	F2	1C	9E	B3	7E	4F	ýb'd&W-}jò ^0
00000200	EE	F5	8E	FB	13	F5	57	2D	7D	2B	6A	F2	1C	3C	A9	55	ið 00 ðW-}+jò <0U
00000210	D3	8C	99	47	C9	6F	A6	57	2D	7D	30	6A	F2	1C	EF	A3	Ó GÉ0 W-}0jò iæ
00000220	C8	41	03	16	F5	77	99	57	2D	7D	2B	6A	F2	1C	0F		EÄ0 ðw W-}+jò 0
00000230	81	B0	D4	11	19	26	85	7E	7E	50	2D	7D	2B	6A	F2	1C	¿Ç00

By applying a single-byte XOR decryption routine, we recovered the plain-text configuration. Figure 5 shows that this reveals the hard-coded command-and-control (C2) URLs and exfiltration paths.

Figure 5. XOR decryption on resource section.

Gremlin stealer uses the resource section to mirror the tactics of several high-profile malware families that frequently use this area for payload obfuscation, including:

- Agent Tesla
- GuLoader
- LokiBot
- Quasar RAT

Comparison with Older Version

Comparing past and present versions reveals a clear evolution in Gremlin stealer's anti-analysis techniques. Legacy samples (shown in Figure 6) lacked obfuscation, leaving function exports and internal symbols intact.

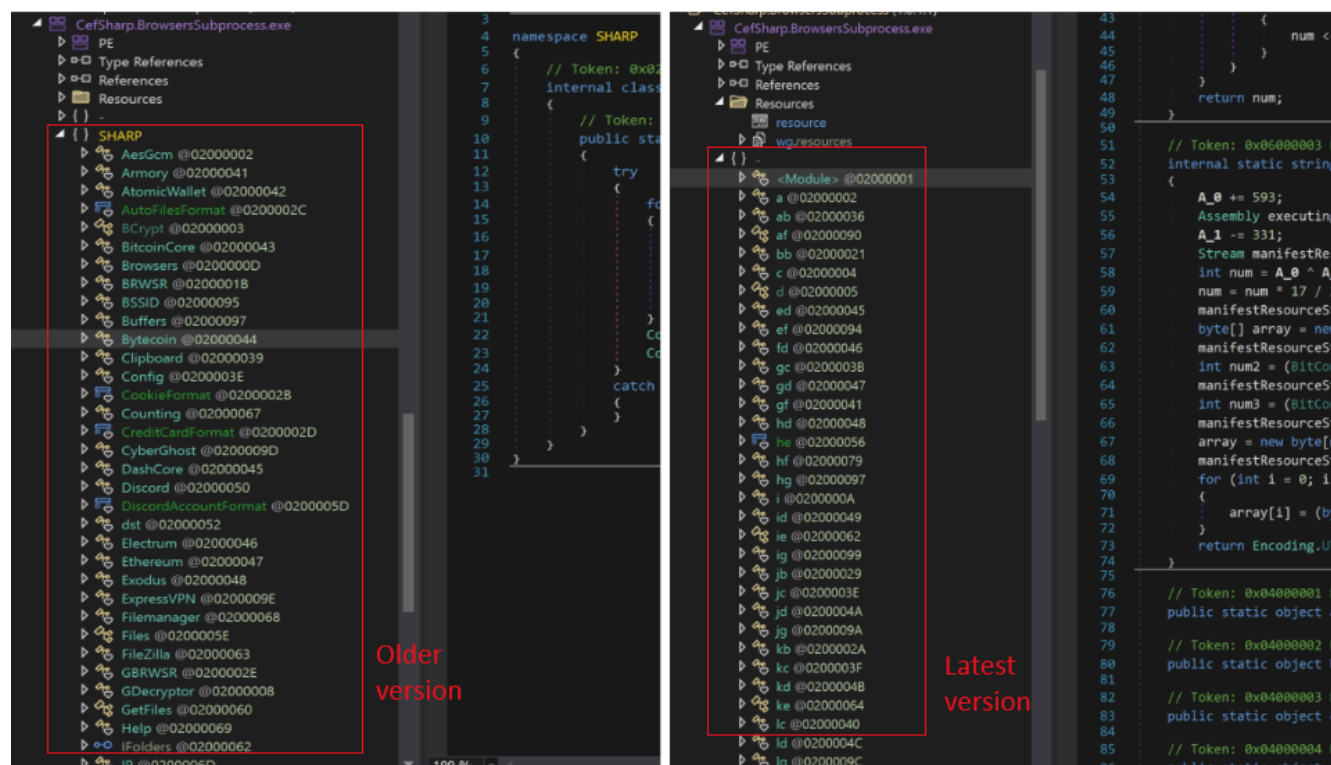


Figure 6. Gremlin older version vs. latest version.

The current iteration implements a staged loading mechanism. Each critical function is decrypted and mapped into memory from the .NET resource section only when needed. This method forces analysts to perform dynamic debugging to observe any meaningful program behavior.

Key Enhancements in the Latest Variant

Gremlin stealer's evolution from a basic credential harvester to a modular toolkit is evident in several key architectural upgrades:

- **Expanded target scope:** Gremlin stealer includes a dedicated Discord token extraction module, which signifies a pivot toward targeting digital identity and social engineering.
- **Active financial fraud:** The latest variant shifts from passive data theft to active financial interference. This crypto clipper functionality continuously monitors the system clipboard for strings matching cryptocurrency wallet patterns. When it detects a match, the malware replaces the victim's address with the attacker's wallet in real time, diverting funds during transactions.
- **Advanced persistence:** The WebSocket-based session hijacking module represents its most significant technical upgrade. This allows Gremlin stealer to hijack active, live browser sessions and bypass modern cookie protections by requesting the data directly from the running browser process.

Sample Packed Using a Complex Commercial Packing Utility

We uncovered an iteration of Gremlin stealer (SHA256 2172dae9a5a695e00e0e4609e7db0207d8566d225f7e815fada246ae995c0f9b) packed using a packing utility, as shown in Figure 7 below.

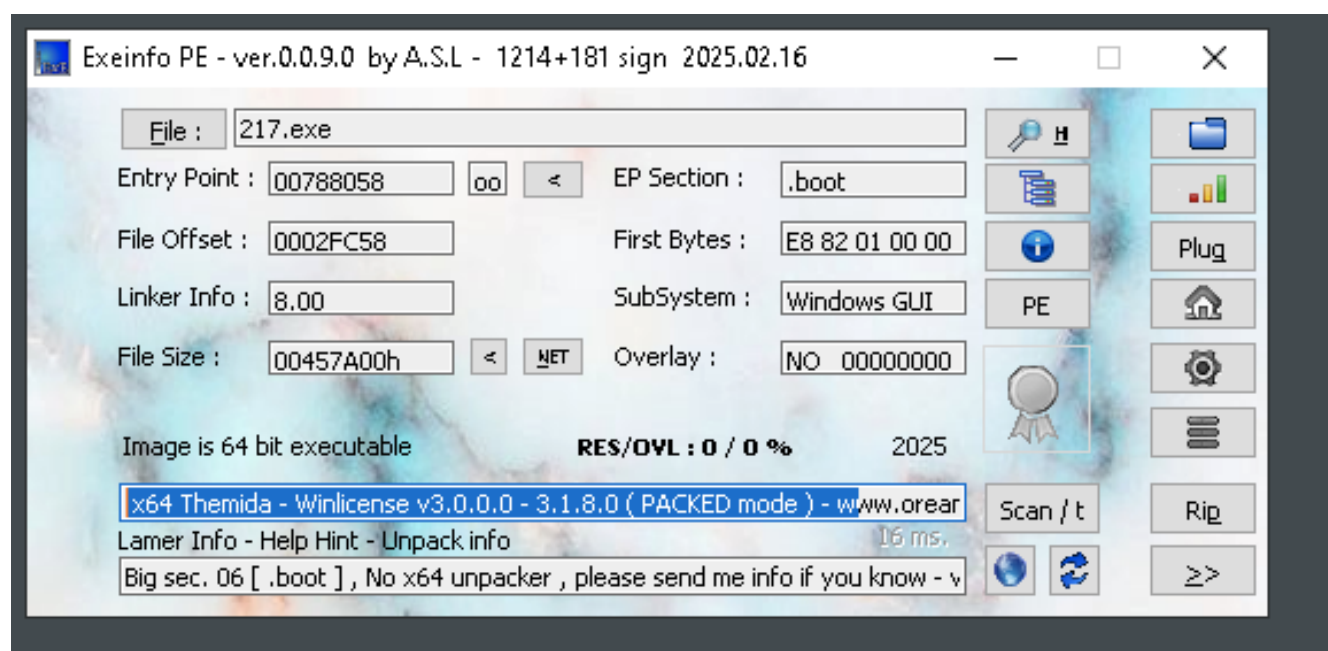


Figure 7. Packed Gremlin variant.

Let's discuss the obfuscation and anti-analysis techniques this variant uses.

Code Obfuscation and Anti Analysis

Identifier Renaming (The “No-Labels” Technique)

Imagine trying to cook in a kitchen where every can, box and spice jar has its label replaced with a random, short name like a, b, c, hf or ze. The malware authors applied this technique to

the variant's code.

- What it is: They replaced every meaningful name for a class, method or variable with a meaningless one.
- Why it's effective: It removes all context. A method originally named `StealPasswordsFromChrome` might become `a()`. A variable named `decryptionKey` might become `b`. This forces an analyst to manually trace every single function call to figure out its purpose, which is incredibly time-consuming.
- Example: In the file `hf.cs`, the main orchestrator class is named `hf`, and its primary methods are `a`, `b` and `c`. In `bb.cs`, the class for stealing browser data is `BrowserCredentialStealer`, but in the original obfuscated code, it was just `bb`.

String Encryption (The “Secret Decoder Ring” Technique)

In this technique, malware authors made readable strings appear as gibberish. Instead of writing a word like `password` or a URL like `hxxps://api[.]telegram[.]org` directly in the code, the malware stores them encrypted.

- What it is: All important strings are hidden. The code only contains numbers that act as a key to a secret decoder function. When the program needs a string, it passes these numbers to the decoder, which then returns the real string.
- The decoder ring function: The secret decoder is the method `_003CModule_003E.c(int, int, int)`.
 - It takes three integers as input
 - It uses these numbers to calculate an offset and a length
 - It opens an embedded resource file (named `resource` in the `.csproj` file) which contains all the encrypted strings
 - It seeks the calculated offset, reads the specified number of encrypted bytes and uses the third integer as a key to decrypt them
 - It returns the final, readable string
- Why it's effective: It completely hides the malware's intentions from static analysis. Analysts cannot simply search the code for suspicious keywords like `Telegram`, `wallet.dat` or `api.ipify[.]org` because they don't exist in plain text. Instead, analysts must either run the program in a debugger to see what strings are produced or reverse-engineer the decoder function.
- Example: A line like this from the original code:

```
csharp
```

```
// This is what the obfuscated code looks like
```

```
string url = global::_003CModule_003E.c(18829, 2178, 23);
```

When executed, the `c()` function would run its decoding operation, and the URL variable would then contain: `csharp`

// This is the real value at runtime

```
string url = "http://api.ipify.org/?format=json";
```

Control-Flow Obfuscation (The “Maze of Useless Roads” Technique)

This technique makes the code's logic intentionally confusing, like turning a straight road into a maze of dead ends and pointless loops that all eventually lead to the same place.

- What it is: The decompiler output is filled with complex and nonsensical if-else statements, goto jumps and mathematical operations that don't actually affect the outcome. These are designed to confuse both human analysts and automated analysis tools.
- Why it's effective: It breaks the logical flow that a person would expect to see. It makes it hard to determine which path the code will *actually* take, even though in many cases, there's only one real path. This significantly increases the time and effort required for reverse engineering.
- Example: There are many switch statements and goto labels (e.g., `IL_00c8`, `IL_0138`) that create a tangled web of execution, even though the underlying logic is a simple sequence of `await Task.Run(...)`.

Conclusion

While the core architecture and exfiltration methods via private web panels or the Telegram Bot API remain consistent, this latest variant of Gremlin stealer represents an evolution into a more complex threat. By transitioning from a simple data exfiltration tool to a more advanced modular stealer, Gremlin now targets Chromium-based browsers. It uses memory-resident techniques to hijack active session tokens and sensitive data directly from running processes, rather than relying solely on static database files.

This threat's scope has broadened, as evidenced by a dedicated Discord token stealer. This module scans multiple paths and uses regex validation to compromise modern communication platforms.

The malware's author has also added a clipboard hijacker. This new monetization feature enables persistent financial fraud. It continuously monitors the clipboard, replacing cryptocurrency wallet addresses with attacker-controlled ones.

Palo Alto Networks Protection and Mitigation

Palo Alto Networks customers are better protected from the threats discussed above through the following products:

- The Advanced WildFire machine-learning models and analysis techniques have been reviewed and updated in light of the IoCs shared in this research.
- Advanced URL Filtering and Advanced DNS Security identify known domains and URLs associated with this activity as malicious.
- Advanced Threat Prevention has an inbuilt machine learning-based detection that can detect exploits in real time.
- Cortex XDR and XSIAM are designed to:
 - Prevent the execution of known malicious malware, and also prevent the execution of unknown malware using Behavioral Threat Protection and machine learning based on the Local Analysis module.
 - Protect against credential gathering tools and techniques using the new Credential Gathering Protection available from Cortex XDR 3.4.
 - Detect post-exploit activity, including credential-based attacks, with behavioral analytics, through Cortex XDR Pro.

If you think you may have been compromised or have an urgent matter, get in touch with the Unit 42 Incident Response team or call:

- North America: Toll Free: +1 (866) 486-4842 (866.4.UNIT42)
- UK: +44.20.3743.3660
- Europe and Middle East: +31.20.299.3130
- Asia: +65.6983.8730
- Japan: +81.50.1790.0200
- Australia: +61.2.4062.7950
- India: 00080005045107

Indicators of Compromise

SHA256 hashes of the Gremlin stealer samples analyzed for this article:

- 2172dae9a5a695e00e0e4609e7db0207d8566d225f7e815fada246ae995c0f9b
- 9aab30a3190301016c79f8a7f8edf45ec088ceecad39926cfcf3418145f3d614
- 971198ff86aeb42739ba9381923d0bc6f847a91553ec57ea6bae5becf80f8759
- ab0fa760bd037a95c4dee431e649e0db860f7cdad6428895b9a399b6991bf3cd
- f76ba1a4650d8cafb6d3ff071688c5db6fd37e165050f03cece693826f51d346
- a9f529a5cbc1f3ee80f785b22e0c472953e6cb226952218aecc7ab07ca328abd

- 691896c7be87e47f3e9ae914d76caaf026aaad0a1034e9f396c2354245215dc3
- 281b970f281dbea3c0e8cfc68b2e9939b253e5d3de52265b454d8f0f578768a2
- 9fda1ddb1acf8dd3685ec31b0b07110855832e3bed28a0f3b81c57fe7fe3ac20
- d11938f14499de03d6a02b5e158782afd903460576e9227e0a15d960a2e9c02c
- 1bd0a200528c82c6488b4f48dd6dbc818d48782a2e25ccd22781c5718c3f62f5

URLs

hxxp[:]194.87.92[.]109/i.php

Additional Resources

Gremlin Stealer: New Stealer on Sale in Underground Forum — Unit 42, Palo Alto Networks